Campus Monterrey

Diseño con lógica programable (Gpo 402)

Maestro:
Raúl Peña Ortega

TE2002B.402

Actividad M4.1 | Diseño de Interfaz de Transmisión UART en VHDL

Omar Andres Garcia Rivera - A00838296
Ángel Enrique Montes Pacheco - A01385688
Ángel Noe Maldonado Ramirez - A01413258

Marzo 26 2026

Código:

Bourate:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity baudrate_gen is
   generic(
      M : integer := 434;  -- Divisor para 115200 baudios con clk de 50MHz
      N : integer := 9     -- Bits necesarios para representar 434
   );
   port(
      clk, reset : in std_logic;
      tick : out std_logic
   );
end baudrate_gen;

architecture behavior of baudrate_gen is
   signal clk_reg : unsigned(N-1 downto 0) := (others => '0');
begin

   process(clk, reset)
   begin
      if reset = '1' then
         clk_reg <= (others => '0');
      elsif rising_edge(clk) then
         if clk_reg = M-1 then
```

```
            clk_reg <= (others => '0');
        else
            clk_reg <= clk_reg + 1;
        end if;
    end if;
  end process;

  -- Genera un pulso (tick) de un ciclo de reloj cada vez que el contador llega a 0
  tick <= '1' when clk_reg = 0 else '0';

end architecture;
```

Uart:

```
uart:library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity uart is
  port(
    s_tick, tx_start, clk, rst : in std_logic;
    d_in : in std_logic_vector(7 downto 0);
    tx, tx_done_tick : out std_logic
  );
end entity;

architecture behavioral of uart is

  type state_type is (idle, start, data, stop);
```

```
    signal C_state, N_state : state_type;

    -- Rango ampliado para evitar errores de "out of range" en síntesis
    signal COUNT : integer range 0 to 15 := 0;
    signal data_reg : std_logic_vector(7 downto 0);

begin

    -- Registro de estado y lógica secuencial
    process(clk, rst)
    begin
        if rst = '1' then
            C_state <= idle;
            COUNT <= 0;
            data_reg <= (others => '0');
        elsif rising_edge(clk) then
            if s_tick = '1' then
                C_state <= N_state;

                if C_state = idle and tx_start = '1' then
                    data_reg <= d_in;
                    COUNT <= 0;
                elsif C_state = data then
                    data_reg <= '0' & data_reg(7 downto 1); -- Desplazamiento a la derecha (LSB
primero)
                    if COUNT < 7 then
                        COUNT <= COUNT + 1;
                    end if;
```

```
            end if;
        end if;
    end if;
end process;

-- Lógica de Próximo Estado (Combinacional)
process(C_state, tx_start, COUNT)
begin
    case C_state is
        when idle =>
            if tx_start = '1' then
                N_state <= start;
            else
                N_state <= idle;
            end if;

        when start =>
            N_state <= data;

        when data =>
            if COUNT = 7 then
                N_state <= stop;
            else
                N_state <= data;
            end if;

        when stop =>
            -- Espera a que se suelte el botón (tx_start = '0') para volver a IDLE
```

```
            -- Esto evita el envío infinito de caracteres
            if tx_start = '0' then
                N_state <= idle;
            else
                N_state <= stop;
            end if;
        end case;
    end process;

    -- Lógica de Salida
    process(C_state, data_reg)
    begin
        case C_state is
            when idle =>
                tx <= '1'; -- Línea en alto (IDLE)
                tx_done_tick <= '0';
            when start =>
                tx <= '0'; -- Bit de inicio
                tx_done_tick <= '0';
            when data =>
                tx <= data_reg(0);
                tx_done_tick <= '0';
            when stop =>
                tx <= '1'; -- Bit de parada
                tx_done_tick <= '1';
        end case;
    end process;
```

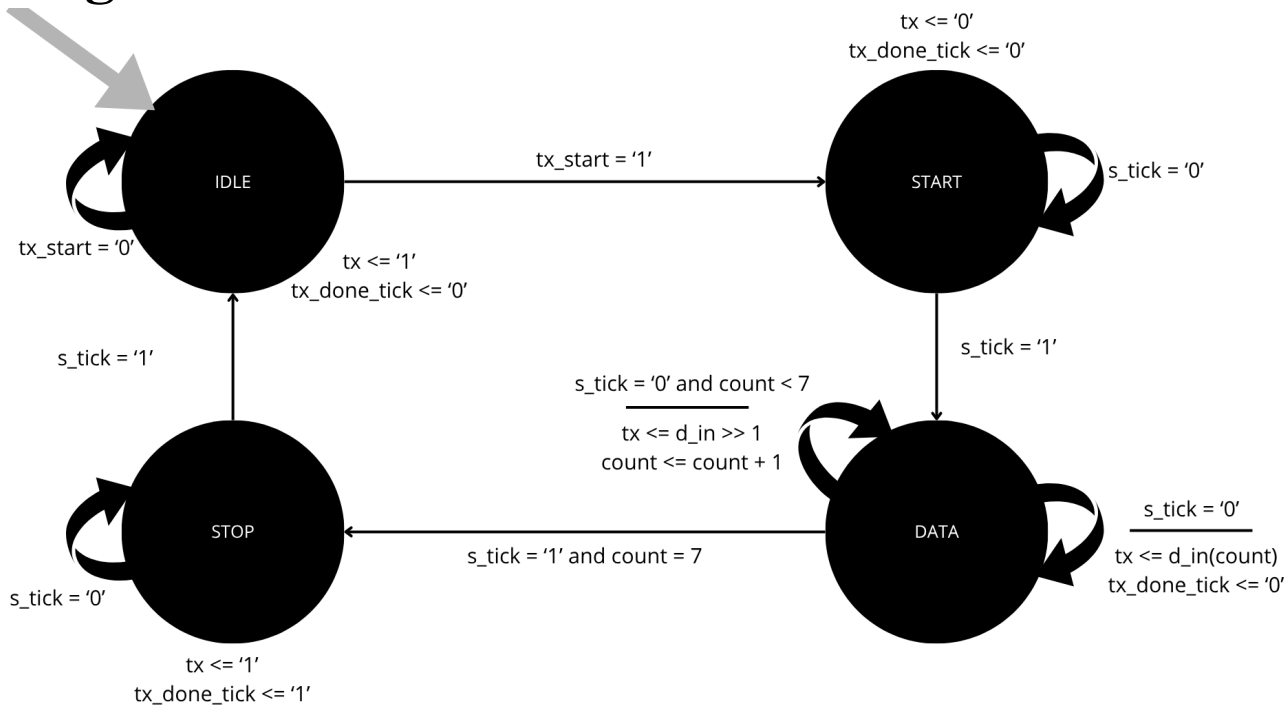
```
end architecture;
```

Top uart:

```
top_uart: library ieee;
use ieee.std_logic_1164.all;

entity top_uart is
   port(
      clk      : in  std_logic;
      rst      : in  std_logic; -- KEY0
      key1     : in  std_logic; -- KEY1
      sw       : in  std_logic_vector(7 downto 0);
      ledr     : out std_logic_vector(7 downto 0);
      tx       : out std_logic
   );
end entity;

architecture rtl of top_uart is
   signal tick        : std_logic;
   signal tx_start    : std_logic;
   signal rst_internal : std_logic;
begin
   ledr <= sw;

   -- CORRECCIÓN DE LÓGICA:
   -- Los botones mandan '0' al presionar.
   -- 'not rst' hace que el reset sea '1' (activo) solo cuando presionas KEY0.
   rst_internal <= not rst;

   -- 'not key1' hace que la transmisión empiece al presionar KEY1.
   tx_start <= not key1;

   baud_unit: entity work.baudrate_gen
      port map(
         clk   => clk,
         reset => rst_internal,
         tick  => tick
      );

   uart_unit: entity work.uart
      port map(
         s_tick      => tick,
         tx_start    => tx_start,
         clk         => clk,
         rst         => rst_internal,
```

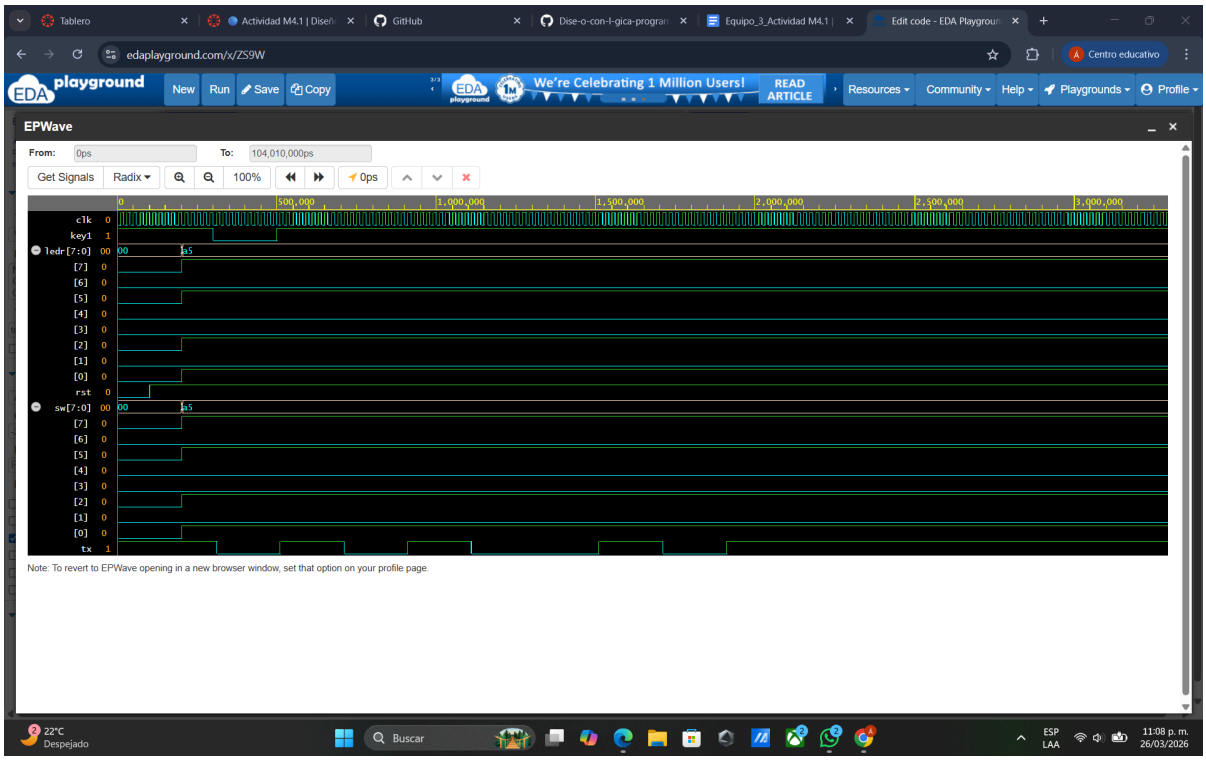
```
        d_in        => sw,
        tx          => tx,
        tx_done_tick => open
    );
end architecture;
```

Diagrama

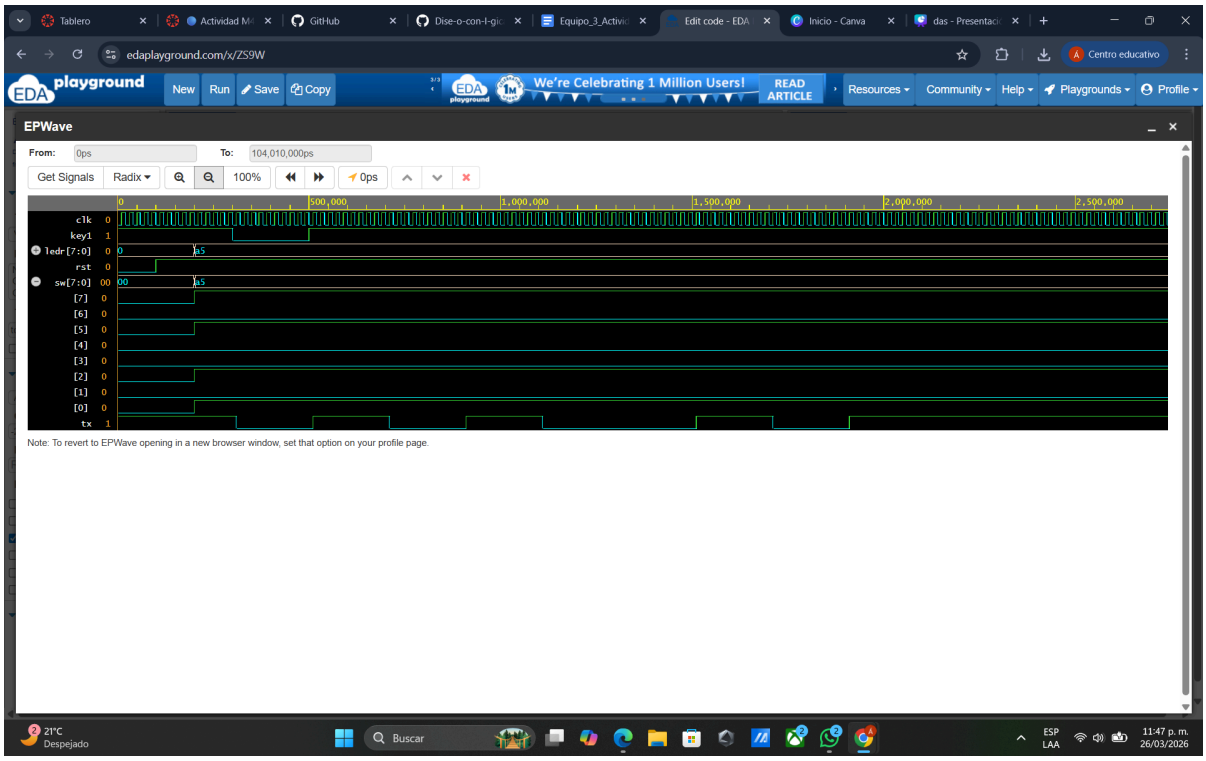


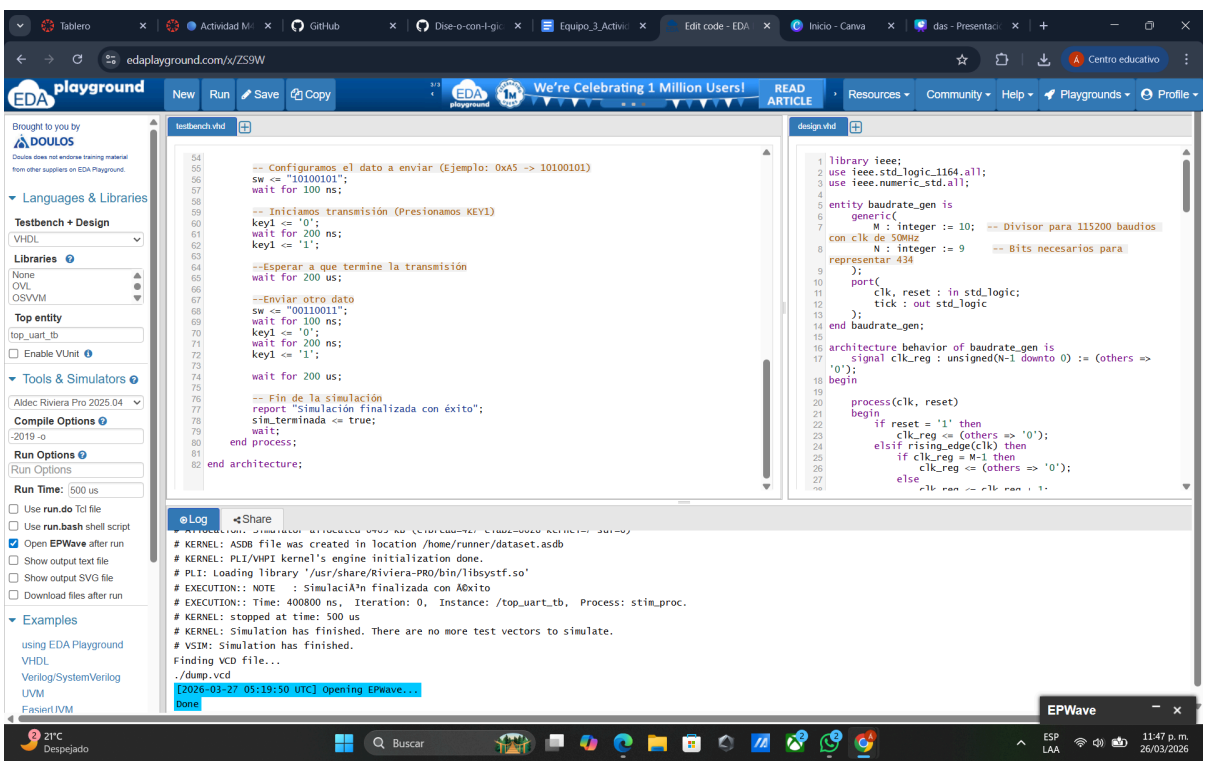
ScreenShots (RTL)

RTL - EDAPlayground



ScreenShots (Pantalla Completa)





TestBench

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity top_uart_tb is
end entity;

architecture sim of top_uart_tb is

   -- Señales para conectar al componente
   signal clk   : std_logic := '0';
   signal rst   : std_logic := '1';
   signal key1  : std_logic := '1';
   signal sw    : std_logic_vector(7 downto 0) := (others => '0');
   signal ledr  : std_logic_vector(7 downto 0);
   signal tx    : std_logic;
   signal sim_terminada : boolean := false;

   -- Constantes de tiempo
   constant CLK_PERIOD : time := 20 ns;

begin

   -- Instancia de la unidad bajo prueba
   uut: entity work.top_uart
      port map(
         clk  => clk,
```

```
      rst  => rst,
      key1 => key1,
      sw   => sw,
      ledr => ledr,
      tx   => tx
   );

-- Generación de Reloj
        clk_process : process
        begin
         while not sim_terminada loop
        clk <= '0'; wait for CLK_PERIOD / 2;
        clk <= '1'; wait for CLK_PERIOD / 2;
                end loop;
         wait; -- Detiene el proceso
        end process;


-- Proceso de Estímulos
stim_proc: process
begin
   -- Reset del sistema (presionamos KEY0 un momento)
   rst <= '0';
   wait for 100 ns;
   rst <= '1';
   wait for 100 ns;

   -- Configuramos el dato a enviar (Ejemplo: 0xA5 -> 10100101)
```

```
        sw <= "10100101";
        wait for 100 ns;

        -- Iniciamos transmisión (Presionamos KEY1)
        key1 <= '0';
        wait for 200 ns;
        key1 <= '1';

        --Esperar a que termine la transmisión
        wait for 200 us;

        --Enviar otro dato
        sw <= "00110011";
        wait for 100 ns;
        key1 <= '0';
        wait for 200 ns;
        key1 <= '1';

        wait for 200 us;

        -- Fin de la simulación
        report "Simulación finalizada con éxito";
        sim_terminada <= true;
                        wait;
    end process;

end architecture;
```

Análisis

En la prueba realizada podemos observar que a partir de la pulsación del botón “KEY 1”, es cuando nosotros iniciamos el traslado o el envió de los datos desde nuestra FPGA, esto lo observamos cuando vemos un estado ‘0’ del “KEY 1”, indicandonos que ha sido presionado, reflejándose en el “tx”, ya que también baja a ‘0’, siendo este nuestro bit de inicio, terminando el traslado de la información gracias al contador interno que cuenta la extensión del mensaje, teniendo como límite 8 bits, esto se ve reflejado en el bit que indica el final de la transmisión de la información, que es un ‘1’, siendo un reflejo claro de cómo es que funciona el protocolo UART.

Si observamos, mantenemos los pines en un estado alto de los bits que queremos trasladar, que en este caso son el 0,2,5,7, iniciando desde la posición 7 del bit, hasta la posición cero, siendo una correcta representación del protocolo UART, ya que envie desde el bit más significativo, hasta el menos significativo.

Video de la funcionalidad completa del Contador en el FPGA:

https://youtu.be/AVt3QN_IQqg